
20.3 Depth-first search

As its name implies, depth-first search searches “deeper” in the graph whenever possible. Depth-first search explores edges out of the most recently discovered vertex v that still has unexplored edges leaving it. Once all of v ’s edges have been explored, the search “backtracks” to explore edges leaving the vertex from which v was discovered. This process continues until all vertices that are reachable from the original source vertex have been discovered. If any undiscovered vertices remain, then depth-first search selects one of them as a new source, repeating the search from that source. The algorithm repeats this entire process until it has discovered every vertex.³

As in breadth-first search, whenever depth-first search discovers a vertex v during a scan of the adjacency list of an already discovered vertex u , it records this event by setting v ’s predecessor attribute $v.\pi$ to u . Unlike breadth-first search, whose predecessor subgraph forms a tree, depth-first search produces a predecessor subgraph that might contain several trees, because the search may repeat from multiple sources. Therefore, we define the *predecessor subgraph* of a depth-first search slightly differently from that of a breadth-first search: it always includes all vertices, and it accounts for multiple sources. Specifically, for a depth-first search the predecessor subgraph is $G_\pi = (V, E_\pi)$, where

$$E_\pi = \{(v.\pi, v) : v \in V \text{ and } v.\pi \neq \text{NIL}\}.$$

The predecessor subgraph of a depth-first search forms a *depth-first forest* comprising several *depth-first trees*. The edges in E_π are *tree edges*.

Like breadth-first search, depth-first search colors vertices during the search to indicate their state. Each vertex is initially white, is grayed when it is *discovered* in the search, and is blackened when it is *finished*, that is, when its adjacency list has been examined completely. This technique guarantees that each vertex ends up in exactly one depth-first tree, so that these trees are disjoint.

Besides creating a depth-first forest, depth-first search also *timestamps* each vertex. Each vertex v has two timestamps: the first timestamp $v.d$ records when v is first discovered (and grayed), and the second timestamp $v.f$ records when the search finishes examining v 's adjacency list (and blackens v). These timestamps provide important information about the structure of the graph and are generally helpful in reasoning about the behavior of depth-first search.

The procedure DFS on the facing page records when it discovers vertex u in the attribute $u.d$ and when it finishes vertex u in the attribute $u.f$. These timestamps are integers between 1 and $2|V|$, since there is one discovery event and one finishing event for each of the $|V|$ vertices. For every vertex u ,

$$u.d < u.f. \quad (20.4)$$

Vertex u is WHITE before time $u.d$, GRAY between time $u.d$ and time $u.f$, and BLACK thereafter. In the DFS procedure, the input graph G may be undirected or directed. The variable *time* is a global variable used for timestamping. Figure 20.4 illustrates the progress of DFS on the graph shown in Figure 20.2 (but with vertices labeled by letters rather than numbers).

DFS(G)

```

1 for each vertex  $u \in G.V$ 
2    $u.color = \text{WHITE}$ 
3    $u.\pi = \text{NIL}$ 
4  $time = 0$ 
5 for each vertex  $u \in G.V$ 
6   if  $u.color == \text{WHITE}$ 
7     DFS-VISIT( $G, u$ )
```

DFS-VISIT(G, u)

```

1  $time = time + 1$            // white vertex  $u$  has just been
                             discovered
2  $u.d = time$ 
3  $u.color = \text{GRAY}$ 
```

```

4for each vertex  $v$  in G.Adj[ $u$ ] // explore each edge  $(u, v)$ 
5    if  $v.color == WHITE$ 
6         $v.\pi = u$ 
7        DFS-VISIT( $G, v$ )
8 $time = time + 1$ 
9 $u.f = time$ 
10 $u.color = BLACK$  // blacken  $u$ ; it is finished

```

The DFS procedure works as follows. Lines 1–3 paint all vertices white and initialize their π attributes to NIL. Line 4 resets the global time counter. Lines 5–7 check each vertex in V in turn and, when a white vertex is found, visit it by calling DFS-VISIT. Upon every call of DFS-VISIT(G, u) in line 7, vertex u becomes the root of a new tree in the depth-first forest. When DFS returns, every vertex u has been assigned a *discovery time* $u.d$ and a *finish time* $u.f$.

In each call DFS-VISIT(G, u), vertex u is initially white. Lines 1–3 increment the global variable $time$, record the new value of $time$ as the discovery time $u.d$, and paint u gray. Lines 4–7 examine each vertex v adjacent to u and recursively visit v if it is white. As line 4 considers each vertex $v \in Adj[u]$, the depth-first search *explores* edge (u, v) . Finally, after every edge leaving u has been explored, lines 8–10 increment $time$, record the finish time in $u.f$, and paint u black.

The results of depth-first search may depend upon the order in which line 5 of DFS examines the vertices and upon the order in which line 4 of DFS-VISIT visits the neighbors of a vertex. These different visitation orders tend not to cause problems in practice, because many applications of depth-first search can use the result from any depth-first search.

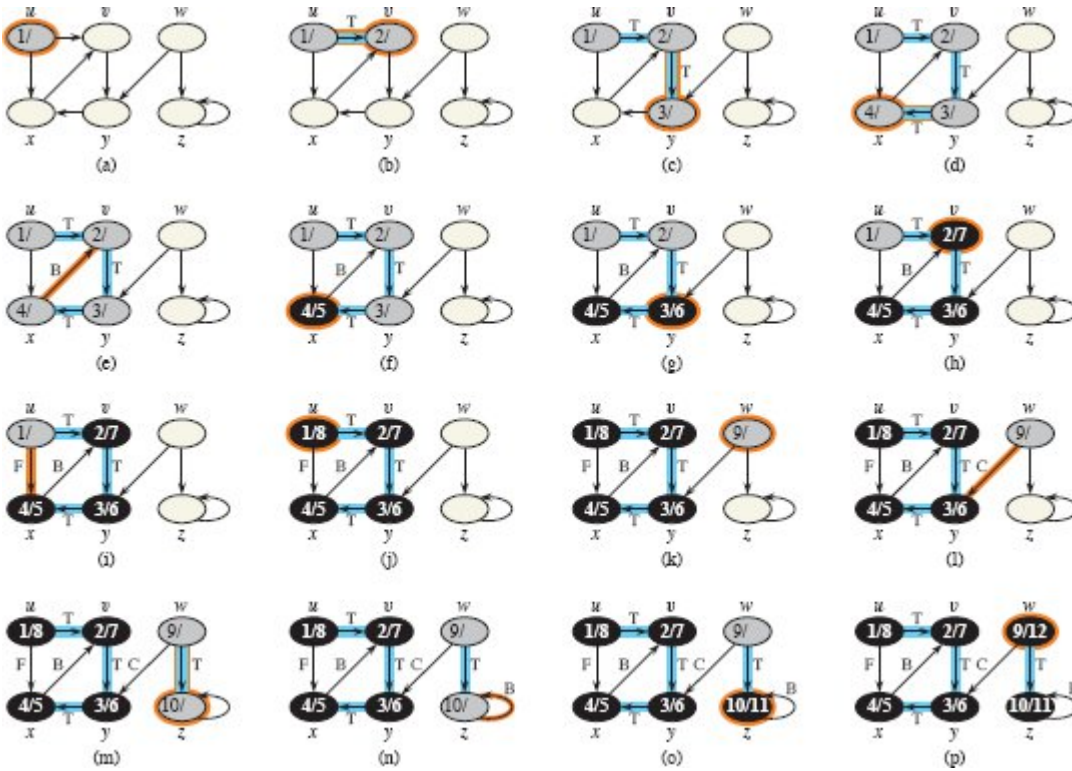


Figure 20.4 The progress of the depth-first-search algorithm DFS on a directed graph. Edges are classified as they are explored: tree edges are labeled T, back edges B, forward edges F, and cross edges C. Timestamps within vertices indicate discovery time/finish times. Tree edges are highlighted in blue. Orange highlights indicate vertices whose discovery or finish times change and edges that are explored in each step.

What is the running time of DFS? The loops on lines 1–3 and lines 5–7 of DFS take $\Theta(V)$ time, exclusive of the time to execute the calls to DFS-VISIT. As we did for breadth-first search, we use aggregate analysis. The procedure DFS-VISIT is called exactly once for each vertex $v \in V$, since the vertex u on which DFS-VISIT is invoked must be white and the first thing DFS-VISIT does is paint vertex u gray. During an execution of DFS-VISIT(G, v), the loop in lines 4–7 executes $|Adj[v]|$ times. Since $\sum_{v \in V} |Adj[v]| = \Theta(E)$ and DFS-VISIT is called once per vertex, the total cost of executing lines 4–7 of DFS-VISIT is $\Theta(V + E)$. The running time of DFS is therefore $\Theta(V + E)$.

Properties of depth-first search

Depth-first search yields valuable information about the structure of a graph. Perhaps the most basic property of depth-first search is that the predecessor subgraph G_π does indeed form a forest of trees, since the structure of the depth-first trees exactly mirrors the structure of recursive calls of DFS-VISIT. That is, $u = v.\pi$ if and only if DFS-VISIT(G, v) was called during a search of u 's adjacency list. Additionally, vertex v is a descendant of vertex u in the depth-first forest if and only if v is discovered during the time in which u is gray.

Another important property of depth-first search is that discovery and finish times have *parenthesis structure*. If the DFS-VISIT procedure were to print a left parenthesis “(u ” when it discovers vertex u and to print a right parenthesis “ u)” when it finishes u , then the printed expression would be well formed in the sense that the parentheses are properly nested. For example, the depth-first search of Figure 20.5(a) corresponds to the parenthesization shown in Figure 20.5(b). The following theorem provides another way to characterize the parenthesis structure.

Theorem 20.7 (Parenthesis theorem)

In any depth-first search of a (directed or undirected) graph $G = (V, E)$, for any two vertices u and v , exactly one of the following three conditions holds:

- the intervals $[u.d, u.f]$ and $[v.d, v.f]$ are entirely disjoint, and neither u nor v is a descendant of the other in the depth-first forest,
- the interval $[u.d, u.f]$ is contained entirely within the interval $[v.d, v.f]$, and u is a descendant of v in a depth-first tree, or
- the interval $[v.d, v.f]$ is contained entirely within the interval $[u.d, u.f]$, and v is a descendant of u in a depth-first tree.

Proof We begin with the case in which $u.d < v.d$. We consider two subcases, according to whether $v.d < u.f$. The first subcase occurs when $v.d < u.f$, so that v was discovered while u was still gray, which implies that v is a descendant of u . Moreover, since v was discovered after u , all of its outgoing edges are explored, and v is finished, before the search returns to and finishes u . In this case, therefore, the interval $[v.d, v.f]$ is

entirely contained within the interval $[u.d, u.f]$. In the other subcase, $u.f < v.d$, and by inequality (20.4), $u.d < u.f < v.d < v.f$, and thus the intervals $[u.d, u.f]$ and $[v.d, v.f]$ are disjoint. Because the intervals are disjoint, neither vertex was discovered while the other was gray, and so neither vertex is a descendant of the other.

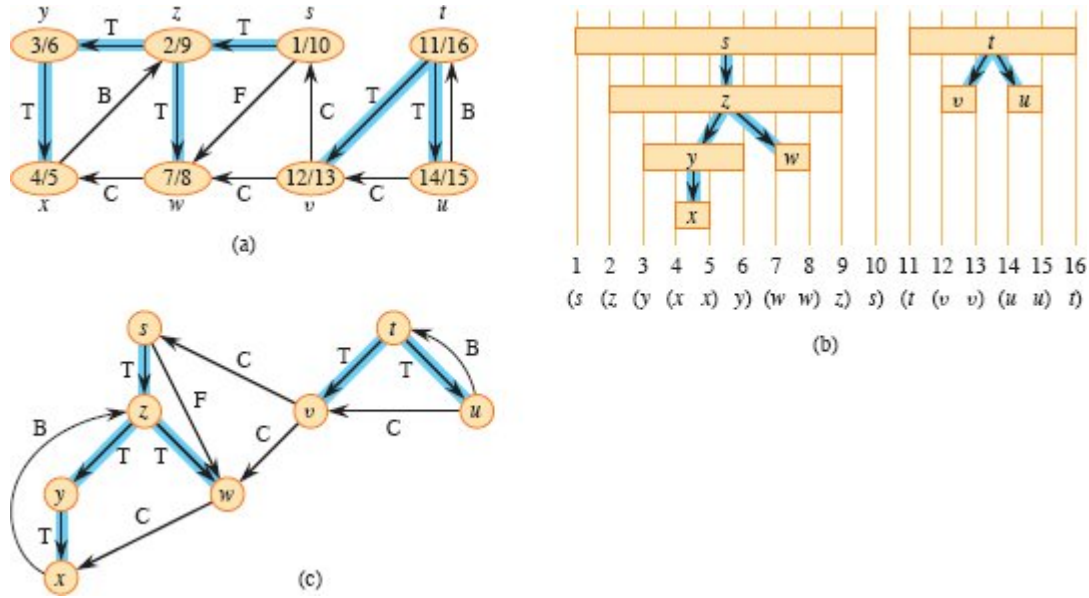


Figure 20.5 Properties of depth-first search. **(a)** The result of a depth-first search of a directed graph. Vertices are timestamped and edge types are indicated as in Figure 20.4. **(b)** Intervals for the discovery time and finish time of each vertex correspond to the parenthesization shown. Each rectangle spans the interval given by the discovery and finish times of the corresponding vertex. Only tree edges are shown. If two intervals overlap, then one is nested within the other, and the vertex corresponding to the smaller interval is a descendant of the vertex corresponding to the larger. **(c)** The graph of part (a) redrawn with all tree and forward edges going down within a depth-first tree and all back edges going up from a descendant to an ancestor.

The case in which $v.d < u.d$ is similar, with the roles of u and v reversed in the above argument. ■

Corollary 20.8 (Nesting of descendants' intervals)

Vertex v is a proper descendant of vertex u in the depth-first forest for a (directed or undirected) graph G if and only if $u.d < v.d < v.f < u.f$.

Proof Immediate from Theorem 20.7.

■

The next theorem gives another important characterization of when one vertex is a descendant of another in the depth-first forest.

Theorem 20.9 (White-path theorem)

In a depth-first forest of a (directed or undirected) graph $G = (V, E)$, vertex v is a descendant of vertex u if and only if at the time $u.d$ that the search discovers u , there is a path from u to v consisting entirely of white vertices.

Proof $\Rightarrow$: If $v = u$, then the path from u to v contains just vertex u , which is still white when $u.d$ receives a value. Now, suppose that v is a proper descendant of u in the depth-first forest. By Corollary 20.8, $u.d < v.d$, and so v is white at time $u.d$. Since v can be any descendant of u , all vertices on the unique simple path from u to v in the depth-first forest are white at time $u.d$.

$\Leftarrow$: Suppose that there is a path of white vertices from u to v at time $u.d$, but v does not become a descendant of u in the depth-first tree. Without loss of generality, assume that every vertex other than v along the path becomes a descendant of u . (Otherwise, let v be the closest vertex to u along the path that doesn't become a descendant of u .) Let w be the predecessor of v in the path, so that w is a descendant of u (w and u may in fact be the same vertex). By Corollary 20.8, $w.f \leq u.f$. Because v must be discovered after u is discovered, but before w is finished, $u.d < v.d < w.f \leq u.f$. Theorem 20.7 then implies that the interval $[v.d, v.f]$ is contained entirely within the interval $[u.d, u.f]$. By Corollary 20.8, v must after all be a descendant of u .

■

Classification of edges

You can obtain important information about a graph by classifying its edges during a depth-first search. For example, Section 20.4 will show that a directed graph is acyclic if and only if a depth-first search yields no “back” edges (Lemma 20.11).

The depth-first forest G_π produced by a depth-first search on graph G can contain four types of edges:

1. **Tree edges** are edges in the depth-first forest G_π . Edge (u, v) is a tree edge if v was first discovered by exploring edge (u, v) .
2. **Back edges** are those edges (u, v) connecting a vertex u to an ancestor v in a depth-first tree. We consider self-loops, which may occur in directed graphs, to be back edges.
3. **Forward edges** are those nontree edges (u, v) connecting a vertex u to a proper descendant v in a depth-first tree.
4. **Cross edges** are all other edges. They can go between vertices in the same depth-first tree, as long as one vertex is not an ancestor of the other, or they can go between vertices in different depth-first trees.

In Figures 20.4 and 20.5, edge labels indicate edge types. Figure 20.5(c) also shows how to redraw the graph of Figure 20.5(a) so that all tree and forward edges head downward in a depth-first tree and all back edges go up. You can redraw any graph in this fashion.

The DFS algorithm has enough information to classify some edges as it encounters them. The key idea is that when an edge (u, v) is first explored, the color of vertex v says something about the edge:

1. WHITE indicates a tree edge,
2. GRAY indicates a back edge, and
3. BLACK indicates a forward or cross edge.

The first case is immediate from the specification of the algorithm. For the second case, observe that the gray vertices always form a linear chain of descendants corresponding to the stack of active DFS-VISIT invocations. The number of gray vertices is 1 more than the depth in the depth-first forest of the vertex most recently discovered. Depth-first search always explores from the deepest gray vertex, so that an edge that reaches another gray vertex has reached an ancestor. The third case handles the remaining possibility. Exercise 20.3-5 asks you to show

that such an edge (u, v) is a forward edge if $u.d < v.d$ and a cross edge if $u.d > v.d$.

According to the following theorem, forward and cross edges never occur in a depth-first search of an undirected graph.

Theorem 20.10

In a depth-first search of an undirected graph G , every edge of G is either a tree edge or a back edge.

Proof Let (u, v) be an arbitrary edge of G , and suppose without loss of generality that $u.d < v.d$. Then, while u is gray, the search must discover and finish v before it finishes u , since v is on u 's adjacency list. If the first time that the search explores edge (u, v) , it is in the direction from u to v , then v is undiscovered (white) until that time, for otherwise the search would have explored this edge already in the direction from v to u . Thus, (u, v) becomes a tree edge. If the search explores (u, v) first in the direction from v to u , then (u, v) is a back edge, since there must be a path of tree edges from u to v .

■

Since (u, v) and (v, u) are really the same edge in an undirected graph, the proof of Theorem 20.10 says how to classify the edge. When searching from a vertex, which must be gray, if the adjacent vertex is white, then the edge is a tree edge. Otherwise, the edge is a back edge.

The next two sections apply the above theorems about depth-first search.

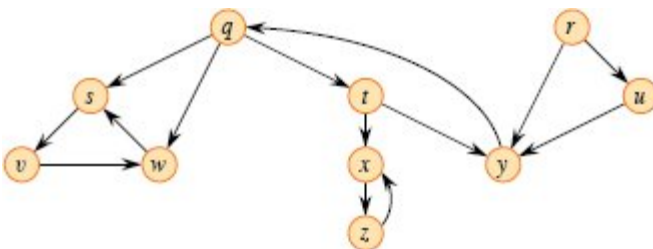


Figure 20.6 A directed graph for use in Exercises 20.3-2 and 20.5-2.

Exercises

20.3-1

Make a 3-by-3 chart with row and column labels WHITE, GRAY, and BLACK. In each cell (i, j) , indicate whether, at any point during a depth-first search of a directed graph, there can be an edge from a vertex of color i to a vertex of color j . For each possible edge, indicate what edge types it can be. Make a second such chart for depth-first search of an undirected graph.

20.3-2

Show how depth-first search works on the graph of Figure 20.6. Assume that the **for** loop of lines 5–7 of the DFS procedure considers the vertices in alphabetical order, and assume that each adjacency list is ordered alphabetically. Show the discovery and finish times for each vertex, and show the classification of each edge.

20.3-3

Show the parenthesis structure of the depth-first search of Figure 20.4.

20.3-4

Show that using a single bit to store each vertex color suffices by arguing that the DFS procedure produces the same result if line 10 of DFS-VISIT is removed.

20.3-5

Show that in a directed graph, edge (u, v) is

- a.* a tree edge or forward edge if and only if $u.d < v.d < v.f < u.f$,
- b.* a back edge if and only if $v.d \leq u.d < u.f \leq v.f$, and
- c.* a cross edge if and only if $v.d < v.f < u.d < u.f$.

20.3-6

Rewrite the procedure DFS, using a stack to eliminate recursion.

20.3-7

Give a counterexample to the conjecture that if a directed graph G contains a path from u to v , and if $u.d < v.d$ in a depth-first search of G , then v is a descendant of u in the depth-first forest produced.

20.3-8

Give a counterexample to the conjecture that if a directed graph G contains a path from u to v , then any depth-first search must result in $v.d \leq u.f$.

20.3-9

Modify the pseudocode for depth-first search so that it prints out every edge in the directed graph G , together with its type. Show what modifications, if any, you need to make if G is undirected.

20.3-10

Explain how a vertex u of a directed graph can end up in a depth-first tree containing only u , even though u has both incoming and outgoing edges in G .

20.3-11

Let $G = (V, E)$ be a connected, undirected graph. Give an $O(V + E)$ -time algorithm to compute a path in G that traverses each edge in E exactly once in each direction. Describe how you can find your way out of a maze if you are given a large supply of pennies.

20.3-12

Show how to use a depth-first search of an undirected graph G to identify the connected components of G , so that the depth-first forest contains as many trees as G has connected components. More precisely, show how to modify depth-first search so that it assigns to each vertex v an integer label $v.cc$ between 1 and k , where k is the number of connected components of G , such that $u.cc = v.cc$ if and only if u and v belong to the same connected component.

★ 20.3-13

A directed graph $G = (V, E)$ is *singly connected* if $u \rightsquigarrow v$ implies that G contains at most one simple path from u to v for all vertices $u, v \in V$. Give an efficient algorithm to determine whether a directed graph is singly connected.

20.4 Topological sort

This section shows how to use depth-first search to perform a topological sort of a directed acyclic graph, or a “dag” as it is sometimes called. A *topological sort* of a dag $G = (V, E)$ is a linear ordering of all its vertices such that if G contains an edge (u, v) , then u appears before v in the ordering. Topological sorting is defined only on directed graphs that are acyclic; no linear ordering is possible when a directed graph contains a cycle. Think of a topological sort of a graph as an ordering of its vertices along a horizontal line so that all directed edges go from left to right. Topological sorting is thus different from the usual kind of “sorting” studied in Part II.

Many applications use directed acyclic graphs to indicate precedences among events. Figure 20.7 gives an example that arises when Professor Bumstead gets dressed in the morning. The professor must don certain garments before others (e.g., socks before shoes). Other items may be put on in any order (e.g., socks and pants). A directed edge (u, v) in the dag of Figure 20.7(a) indicates that garment u must be donned before garment v . A topological sort of this dag therefore gives a possible order for getting dressed. Figure 20.7(b) shows the topologically sorted dag as an ordering of vertices along a horizontal line such that all directed edges go from left to right.

The procedure TOPOLOGICAL-SORT topologically sorts a dag. Figure 20.7(b) shows how the topologically sorted vertices appear in reverse order of their finish times.

TOPOLOGICAL-SORT(G)

- 1 call DFS(G) to compute finish times $v.f$ for each vertex v
- 2 as each vertex is finished, insert it onto the front of a linked list
- 3 **return** the linked list of vertices

The TOPOLOGICAL-SORT procedure runs in $\Theta(V + E)$ time, since depth-first search takes $\Theta(V + E)$ time and it takes $O(1)$ time to insert each of the $|V|$ vertices onto the front of the linked list.